

LAPORAN PRAKTIKUM STRUKTUR DATA
REPRESENTASI BINARY TREE DAN GRAP TRAVERSAL PADA JAVA

Disusun Oleh :

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi, S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 9 dengan judul “Representasi Binary Tree dan Graph Traversal Pada Java” ini dengan baik. Pada laporan praktikum pekan ini akan membahas mengenai bagaimana Implementasi dari Binary Tree pada Java dan Juga Graph Traversal baik itu metode BFS dan DFS.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Averro yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 NODE.....	2
2.1.1 Penjelasan Program.....	2
2.1.2 Kode Program	2
2.2 Binary Tree.....	3
2.2.1 Penjelasan Program.....	3
2.2.2 Kode Program	3
2.3 Binary Tree Driver	5
2.3.1 Penjelasan Program.....	5
2.3.2 Kode Program	6
2.4 Graph Traversal.....	7
2.4.1 Penjelasan Program.....	7
2.4.2 Kode Program	8
BAB III PENUTUP	10
TUGAS PRAKTIKUM.....	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data non-linear merupakan elemen krusial dalam ilmu komputer untuk merepresentasikan hubungan data yang kompleks dan saling bercabang, yang tidak dapat diakomodasi oleh struktur linear seperti Array atau Linked List. Dua model non-linear yang paling sering digunakan adalah Pohon Biner (Binary Tree) dan Graf (Graph).

Pohon Biner mengorganisasikan data secara hierarkis dengan batasan maksimal dua anak per simpul, yang sangat efisien untuk proses pencarian dan pengurutan data melalui metode traversal (Inorder, Preorder, Postorder). Di sisi lain, Graf menyediakan fleksibilitas penuh untuk memodelkan jaringan hubungan antar-objek tanpa batasan hierarki tertentu. Memahami implementasi dan teknik penelusuran kedua struktur data ini seperti Depth-First Search (DFS) dan Breadth-First Search (BFS) sangat penting bagi mahasiswa untuk membangun algoritma pencarian, analisis jaringan, dan optimasi sistem yang efisien.

1.2 Tujuan Praktikum

- Memahami konsep dasar, representasi memori, dan struktur hierarki dari Pohon Biner (Binary Tree) serta Graf (Graph) dalam pemrograman berorientasi objek menggunakan Java..
- Menguasai mekanisme penelusuran graf (Graph Traversal) menggunakan strategi eksplorasi mendalam (Depth-First Search / DFS) dan eksplorasi melebar (Breadth-First Search / BFS).

1.3 Manfaat Praktikum

Praktikum ini memberikan keterampilan praktis dalam mentransformasikan konsep teoretis struktur data non-linear ke dalam kode program yang siap pakai. Mahasiswa dapat memanfaatkan struktur *Binary Tree* untuk kebutuhan optimasi hierarki data, serta menerapkan algoritma DFS dan BFS.

BAB II

PEMBAHASAN

2.1 NODE

2.1.1 Penjelasan Program

kelas Node yang berfungsi sebagai representasi dasar atau fondasi dari sebuah simpul tunggal di dalam struktur data pohon biner (*Binary Tree*). Setiap objek dari kelas ini dirancang untuk menyimpan sebuah nilai bertipe data integer (`data_2008`) serta memiliki dua referensi penunjuk (*pointer*), yaitu `left_2008` untuk menunjuk ke anak sebelah kiri dan `right_2008` untuk menunjuk ke anak sebelah kanan. Selain menyediakan metode enkapsulasi standar seperti *getter* dan *setter* untuk memanipulasi data dan hubungan antar-simpul secara aman, kelas ini juga memuat logika rekursif inti untuk tiga metode penelusuran pohon biner, yaitu *Preorder* (Root-Kiri-Kanan), *Inorder* (Kiri-Root-Kanan), dan *Postorder* (Kiri-Kanan-Root). Lebih jauh lagi, kelas ini dilengkapi dengan metode `print_2008()` yang memanfaatkan algoritma rekursif string untuk memvisualisasikan bentuk hierarki.

2.1.2 Kode Program

```
void printPreorder_2008 (Node_2511532008 node_2008) {
    if (node_2008 == null)
        return;
    System.out.print(node_2008.data_2008 + " ");
    printPreorder_2008 (node_2008.left_2008);
    printPreorder_2008 (node_2008.right_2008);
}

void printPostorder_2008 (Node_2511532008 node_2008) {
    if (node_2008 == null)
        return;
    printPostorder_2008 (node_2008.left_2008);
    printPostorder_2008 (node_2008.right_2008);
    System.out.print(node_2008.data_2008 + " ");
}

void printInorder_2008 (Node_2511532008 node_2008) {
    if (node_2008 == null)
        return;
    printInorder_2008 (node_2008.left_2008);
    System.out.print(node_2008.data_2008 + " ");
    printInorder_2008 (node_2008.right_2008);
}

public String print_2008() {
    return this.print_2008("", true, "");
}

public String print_2008(String prefix_2008, boolean isTail_2008, String sb_2008) {
    if (right_2008 != null) {
        right_2008.print_2008(prefix_2008 + (isTail_2008 ? "|" : " "), false, sb_2008);
    }
    System.out.println(prefix_2008 + (isTail_2008 ? "\\--" : "/--") + data_2008);
    if (left_2008 != null) {
        left_2008.print_2008(prefix_2008 + (isTail_2008 ? " " : "|"), true, sb_2008);
    }
    return sb_2008;
}

}
```

Gambar 2.1

2.2 Binary Tree

2.2.1 Penjelasan Program

Kelas BTree_2511532008 yang bertindak sebagai struktur pengelola utama bagi keseluruhan pohon biner. Di dalam kelas ini, terdapat atribut root_2008 yang memegang kendali atas simpul akar paling atas dan atribut currentNode_2008 untuk mencatat posisi simpul yang sedang diakses saat ini. Kelas ini menyediakan berbagai fungsionalitas manajerial untuk operasi pohon biner, seperti fungsi search_2008() untuk mencari keberadaan suatu nilai secara rekursif di seluruh cabang, fungsi isEmpty_2008() untuk memeriksa status kekosongan pohon, serta fungsi countNodes_2008() yang bekerja menghitung total seluruh simpul yang terhubung dalam pohon dengan menjumlahkan simpul aktif beserta seluruh anak di sub-pohon kiri dan kanannya.

2.2.2 Kode Program

```
package pekan9_2511532008;
public class BTree_2511532008 {
    private Node_2511532008 root_2008;
    private Node_2511532008 currentNode_2008;
    public BTree_2511532008 () {
        root_2008 = null; }
    public boolean search_2008 (int data_2008) {
        return search_2008 (root_2008, data_2008);
    }
    private boolean search_2008 (Node_2511532008 node_2008 , int data_2008) {
        if (node_2008.getData_2008() == data_2008)
            return true;
        if (node_2008.getLeft_2008() != null)
            if (search_2008(node_2008.getLeft_2008(), data_2008))
                return true;
        if (node_2008.getRight_2008() != null)
            if (search_2008(node_2008.getRight_2008(), data_2008))
                return true;
        return false;
    }
    public void printInorder_2008 () {
        root_2008.printInorder_2008(root_2008);
    }
    public void printPreorder_2008 () {
        root_2008.printPreorder_2008(root_2008);
    }
    public void printPostorder_2008 () {
        root_2008.printPostorder_2008(root_2008);
    }
    public Node_2511532008 getRoot_2008 () {
        return root_2008;
    }
    public boolean isEmpty_2008 () {
        return root_2008 == null;
    }
    public int countNodes_2008 () {
        return countNodes_2008 (root_2008);
    }
}
```

Gambar 2.2

```

private int countNodes_2008 (Node_2511532008 node_2008) {
    int count_2008 = 1;
    if (node_2008 == null){
        return 0;
    } else {
        count_2008 += countNodes_2008 (node_2008.getLeft_2008());
        count_2008 += countNodes_2008 (node_2008.getRight_2008());
        return count_2008; }
}
public void print_2008 () {
    root_2008.print_2008();
}
public Node_2511532008 getCurrent_2008 () {
    return currentNode_2008;
}
public Node_2511532008 setCurrent_2008 (Node_2511532008 node_2008) {
    return currentNode_2008 = node_2008;
}
public void setRoot_2008 (Node_2511532008 root_2008) {
    this.root_2008 = root_2008;
}
}

```

Gambar 2.3

2.3 Binary Tree Driver

2.3.1 Penjelasan Program

Kelas penggerak utama (*main class/driver*) yang berfungsi untuk menguji dan mendemonstrasikan seluruh kapabilitas dari struktur data pohon biner yang telah dibangun sebelumnya. Di dalam metode *main*, program secara berurutan melakukan instansiasi objek *BTree_2511532008* baru, membuat sembilan buah objek simpul independen dari kelas *Node_2511532008* dengan nilai 1 sampai 9, lalu menyusun hubungan hierarkinya secara manual menggunakan perintah *setLeft_2008* dan *setRight_2008* hingga membentuk satu kesatuan pohon biner yang utuh. Setelah struktur pohon terbentuk, kelas *driver* ini melakukan serangkaian pengujian terukur yang hasilnya langsung dicetak ke layar, meliputi verifikasi jumlah simpul pada setiap tahapan konstruksi, pelacakan nilai simpul aktif melalui *current node*, pembuktian kebenaran hasil urutan dari ketiga algoritma *traversal (Inorder, Preorder, Postorder)*, serta pencetakan representasi skema visual pohon.

2.3.2 Kode Program

```
package pekan9_2511532008;

public class BtreeDriver_2511532008 {
    public static void main (String [] args) {

        BTree_2511532008 tree_2008 = new BTree_2511532008 ();
        System.out.print("Jumlah Simpul Awal Pohon : ");
        System.out.println(tree_2008.countNodes_2008());

        //menambahkan simpul data 1
        Node_2511532008 root_2008 = new Node_2511532008 (1);

        //menjadikan simpul 1 sebagai root
        tree_2008.setRoot_2008(root_2008);
        System.out.print("Jumlah simpul jika hanya ada root :");
        System.out.println(tree_2008.countNodes_2008());
        Node_2511532008 node2_2008 = new Node_2511532008 (2);
        Node_2511532008 node3_2008 = new Node_2511532008 (3);
        Node_2511532008 node4_2008 = new Node_2511532008 (4);
        Node_2511532008 node5_2008 = new Node_2511532008 (5);
        Node_2511532008 node6_2008 = new Node_2511532008 (6);
        Node_2511532008 node7_2008 = new Node_2511532008 (7);
        Node_2511532008 node8_2008 = new Node_2511532008 (8);
        Node_2511532008 node9_2008 = new Node_2511532008 (9);

        root_2008.setLeft_2008(node2_2008);
        node2_2008.setLeft_2008(node4_2008);
        node2_2008.setRight_2008(node5_2008);
        node4_2008.setRight_2008(node8_2008);
        root_2008.setRight_2008(node3_2008);
        node3_2008.setLeft_2008(node6_2008);
        node3_2008.setRight_2008(node7_2008);
        node6_2008.setLeft_2008(node9_2008);

        tree_2008.setCurrent_2008(tree_2008.getRoot_2008());
        System.out.println("menampilkan simpul terakhir :");
        System.out.println(tree_2008.getCurrent_2008().getData_2008());
        System.out.println("Jumlah simpul ; setelah simpul 7 ditambahkan :");
        System.out.println(tree_2008.countNodes_2008());
        System.out.println("InOrder:");
        tree_2008.printInorder_2008();
        System.out.println("\nPreOrder:");
        tree_2008.printPreorder_2008();
        System.out.println("\nPostOrder:");
        tree_2008.printPostorder_2008();
        System.out.println("\nmenampilkan simpul dalam bentuk phon : ");
        tree_2008.print_2008();
    }
}
```

Gambar 2.4

2.4 Graph Traversal

2.4.1 Penjelasan Program

File terakhir ini mengimplementasikan kelas `GraphTraversal_2511532008` yang berfokus pada pemodelan serta penerapan algoritma penelusuran pada struktur data Graf Tak Berarah (*Undirected Graph*). Berbeda dengan pohon biner, hubungan antar-simpul (*vertex*) di dalam graf ini direpresentasikan menggunakan konsep *Adjacency List* (Daftar Ketetanggaan) yang dibangun di atas struktur data `Map<String, List<String>>`, di mana setiap simpul dipetakan ke sebuah *ArrayList* yang berisi daftar simpul tetangga yang terhubung langsung dengannya melalui metode `addEdge_2008()`. Kelas ini mendemonstrasikan dua algoritma penelusuran graf fundamental di dalam metode utamanya: pertama adalah *Depth-First Search* (DFS) yang mengeksplorasi cabang graf sedalam mungkin ke bawah secara rekursif menggunakan pelacak `Set` untuk menghindari perulangan, dan kedua adalah *Breadth-First Search* (BFS) yang menjelajahi graf secara melebar per tingkat jarak memanfaatkan bantuan struktur data *Antrean* (*Queue*) secara iteratif.

2.4.2 Kode Program

```
package pekan9_2511532008;
import java.util.*;

public class GraphTraversal_2511532008 {
    private Map <String, List <String>> graph_2008 = new HashMap <> ();
    public void addEdge_2008 (String node1_2008, String node2_2008) {
        graph_2008.putIfAbsent(node1_2008, new ArrayList<>());
        graph_2008.putIfAbsent(node2_2008, new ArrayList<>());
        graph_2008.get(node1_2008).add(node2_2008);
        graph_2008.get(node2_2008).add(node1_2008);
    }
    public void printGraph_2008() {
        System.out.println("Graf Awal (Adjacency List) : ");
        for (String node_2008 : graph_2008.keySet()) {
            System.out.print(node_2008 + "-> ");
            List<String> neighbors_2008 = graph_2008.get(node_2008);
            System.out.println(String.join(", ", neighbors_2008));
        }
        System.out.println();
    }
    public void dfs_2008 (String start_2008) {
        Set <String> visited_2008 = new HashSet <>();
        System.out.println("Penelusuran DFS : ");
        dfsHelper_2008(start_2008, visited_2008);
        System.out.println();
    }
    private void dfsHelper_2008 (String current_2008, Set <String> visited_2008) {
        if (visited_2008.contains(current_2008)) return;
        visited_2008.add(current_2008);
        System.out.print(current_2008 + " ");
        for (String neighbor_2008 : graph_2008.getDefault(current_2008, new ArrayList<>())) {
            dfsHelper_2008(neighbor_2008, visited_2008);
        }
    }
}
```

Gambar 2.5

```

void bfs_2008 (String start_2008) {
    Set<String> visited_2008 = new HashSet <>();
    Queue <String> queue_2008 = new LinkedList<>();
    queue_2008.add(start_2008);
    visited_2008.add(start_2008);
    System.out.println("Penelusuran BFS :");
    while (!queue_2008.isEmpty()) {
        String current_2008 = queue_2008.poll();
        System.out.print(current_2008 + " ");
        for (String neighbor_2008 : graph_2008.getOrDefault(current_2008, new ArrayList<>())) {
            if (!visited_2008.contains(neighbor_2008)) {
                queue_2008.add(neighbor_2008);
                visited_2008.add(neighbor_2008);
            }
        }
    }
    System.out.println();
}

public static void main (String [] Args) {
    GraphTraversal_2511532008 graph_2008 = new GraphTraversal_2511532008();

    graph_2008.addEdge_2008("A", "B");
    graph_2008.addEdge_2008("A", "C");
    graph_2008.addEdge_2008("B", "D");
    graph_2008.addEdge_2008("B", "E");

    System.out.println("Graf Awal adalah : ");
    graph_2008.printGraph_2008();

    //lakukan penelusuran
    graph_2008.dfs_2008("A");
    graph_2008.bfs_2008("A");
}
}

```

Gambar 2.6

BAB III

PENUTUP

Praktikum ini berhasil mengimplementasikan struktur data non-linear Pohon Biner (Binary Tree) dan Graf (Graph) menggunakan bahasa pemrograman Java dengan pendekatan berorientasi objek. Melalui pembuatan kelas pohon biner, dapat disimpulkan bahwa pengorganisasian data secara hierarkis sangat bergantung pada pemanfaatan fungsi rekursif, baik untuk memanipulasi hubungan antar-simpul, menghitung jumlah node, maupun dalam mengeksekusi tiga metode traversal (Inorder, Preorder, Postorder). Sementara itu, melalui implementasi objek graf berbasis Adjacency List, praktikum ini membuktikan karakteristik dua algoritma penelusuran fundamental: Depth-First Search (DFS) yang efektif untuk eksplorasi mendalam secara rekursif, dan Breadth-First Search (BFS) yang unggul dalam eksplorasi melebar per tingkat jarak memanfaatkan bantuan struktur data antrean (Queue).

TUGAS PRAKTIKUM

```
public class PetaKampus_2511532008 extends JFrame {  
  
    static class Node_2008 {  
        String name_2008;  
        int x_2008, y_2008;  
  
        Node_2008(String name_2008, int x_2008, int y_2008) {  
            this.name_2008 = name_2008;  
            this.x_2008 = x_2008;  
            this.y_2008 = y_2008;  
        }  
    }  
}
```

```

private void addEdge_2008(int u_2008, int v_2008) {
    adjList_2008.get(u_2008).add(v_2008);
    adjList_2008.get(v_2008).add(u_2008);
}
public void BFS() {
    int startIdx_2008 = startCombo_2008.getSelectedIndex();
    int goalIdx_2008 = goalCombo_2008.getSelectedIndex();

    visitedIndices_2008.clear();
    visitedOrder_2008.clear();
    finalPath_2008.clear();

    Queue<Integer> queue_2008 = new LinkedList<>();
    Map<Integer, Integer> parentMap_2008 = new HashMap<>();

    queue_2008.add(startIdx_2008);
    visitedIndices_2008.add(startIdx_2008);
    visitedOrder_2008.add(startIdx_2008);

    boolean found_2008 = false;

    while (!queue_2008.isEmpty()) {
        int current_2008 = queue_2008.poll();

        if (current_2008 == goalIdx_2008) {
            found_2008 = true;
            break;
        }

        for (int neighbor_2008 : adjList_2008.get(current_2008)) {
            if (!visitedIndices_2008.contains(neighbor_2008)) {
                visitedIndices_2008.add(neighbor_2008);
                visitedOrder_2008.add(neighbor_2008);
                parentMap_2008.put(neighbor_2008, current_2008);
                queue_2008.add(neighbor_2008);
            }
        }
    }

    if (found_2008) {
        int curr_2008 = goalIdx_2008;
        while (curr_2008 != startIdx_2008) {
            finalPath_2008.add(0, curr_2008);
            curr_2008 = parentMap_2008.get(curr_2008);
        }
        finalPath_2008.add(0, startIdx_2008);
    }

    displayPath();
    displayGraph();
}

```

```

public void DFS() {
    int startIdx_2008 = startCombo_2008.getSelectedIndex();
    int goalIdx_2008 = goalCombo_2008.getSelectedIndex();

    visitedIndices_2008.clear();
    visitedOrder_2008.clear();
    finalPath_2008.clear();

    Stack<Integer> stack_2008 = new Stack<>();
    Map<Integer, Integer> parentMap_2008 = new HashMap<>();

    stack_2008.push(startIdx_2008);

    boolean found_2008 = false;

    while (!stack_2008.isEmpty()) {
        int current_2008 = stack_2008.pop();

        if (!visitedIndices_2008.contains(current_2008)) {
            visitedIndices_2008.add(current_2008);
            visitedOrder_2008.add(current_2008);

            if (current_2008 == goalIdx_2008) {
                found_2008 = true;
                break;
            }

            List<Integer> neighbors_2008 = adjList_2008.get(current_2008);
            for (int i_2008 = neighbors_2008.size() - 1; i_2008 >= 0; i_2008--) {
                int neighbor_2008 = neighbors_2008.get(i_2008);
                if (!visitedIndices_2008.contains(neighbor_2008)) {
                    stack_2008.push(neighbor_2008);
                    parentMap_2008.put(neighbor_2008, current_2008);
                }
            }
        }
    }

    if (found_2008) {
        int curr_2008 = goalIdx_2008;
        while (curr_2008 != startIdx_2008) {
            finalPath_2008.add(0, curr_2008);
            curr_2008 = parentMap_2008.get(curr_2008);
        }
        finalPath_2008.add(0, startIdx_2008);
    }

    displayPath();
    displayGraph();
}

```

```

public void displayGraph() {
    canvas_2008.repaint();
}

public void displayPath() {
    StringBuilder sb_2008 = new StringBuilder();
    sb_2008.append("Hasil Pencarian :\n");
    sb_2008.append("Jalur : ");
    if (finalPath_2008.isEmpty()) {
        sb_2008.append("Tidak ditemukan jalur\n");
    } else {
        for (int i_2008 = 0; i_2008 < finalPath_2008.size(); i_2008++) {
            sb_2008.append(nodeList_2008.get(finalPath_2008.get(i_2008)).name_2008);
            if (i_2008 < finalPath_2008.size() - 1) {
                sb_2008.append(" -> ");
            }
        }
        sb_2008.append("\n");
    }

    sb_2008.append("Node Dikunjungi : ");
    for (int i_2008 = 0; i_2008 < visitedOrder_2008.size(); i_2008++) {
        sb_2008.append(nodeList_2008.get(visitedOrder_2008.get(i_2008)).name_2008);
        if (i_2008 < visitedOrder_2008.size() - 1) {
            sb_2008.append(", ");
        }
    }
    sb_2008.append("\n");

    sb_2008.append("Jumlah Node Dikunjungi : ").append(visitedOrder_2008.size()).append("\n");
    resultArea_2008.setText(sb_2008.toString());
}

public void resetGraph() {
    visitedIndices_2008.clear();
    visitedOrder_2008.clear();
    finalPath_2008.clear();
    resultArea_2008.setText("Hasil Pencarian :\nJalur :\nNode Dikunjungi :\nJumlah Node Dikunjungi : 0");
    displayGraph();
}

private class GraphPanel_2008 extends JPanel {
    public GraphPanel_2008() {
        setBackground(Color.WHITE);
        setBorder(BorderFactory.createTitledBorder("VISUALISASI GRAPH"));
    }
}

```

```

    private boolean isEdgeInPath_2008(int u_2008, int v_2008) {
        for (int i_2008 = 0; i_2008 < finalPath_2008.size() - 1; i_2008++) {
            int pU_2008 = finalPath_2008.get(i_2008);
            int pV_2008 = finalPath_2008.get(i_2008 + 1);
            if ((pU_2008 == u_2008 && pV_2008 == v_2008) || (pU_2008 == v_2008 && pV_2008 == u_2008)) {
                return true;
            }
        }
        return false;
    }
}

public static void main(String[] args_2008) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new PetaKampus_2511532008().setVisible(true);
        }
    });
}

```

